# Mini Compiler Project


**Session: 2023 – 2027**

**Submitted by:**

Maryam Fatima 2023-CS-61

**Supervised by:**

Ma’am Aliya Farooq

**Course:**

CS-471L Compiler Construction Lab

Department of Computer Science

**University of Engineering and Technology**

**Lahore Pakistan**

# Table of Contents


1. Introduction ..... 3

1.1 Project Objectives ..... 3

1.2 Language Subset Supported ..... 3

2. System Architecture and File Structure ..... 4

2.1 File-to-Module Mapping ..... 4

2.2 Compilation Pipeline ..... 5

3. Module 1: common.h (Shared Definitions) ..... 6

3.1 Token Structure ..... 6

3.2 ASTNode and ASTNodeKind ..... 7

3.3 ErrorHandler ..... 7

3.4 Utility Functions ..... 7

4. Module 2 : lexer.h (Lexical Analyzer) ..... 8

4.1 Double-Buffer Architecture ..... 8

4.2 Scanning Methods ..... 8

4.3 Keyword-to-Token Type Mapping ..... 8

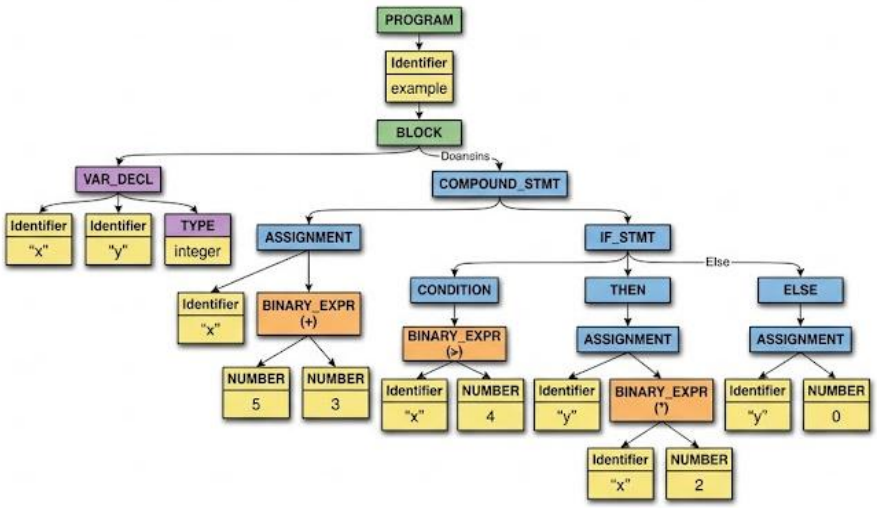
5. Module 3 : recursive_descent_parser.h ..... 9

5.1 Grammar Coverage ..... 9

5.2 Left-Recursion Elimination ..... 9

5.3 AST Construction ..... 9

5.4 Example AST ..... 10




..... 10

5.5 Symbol Table Integration ..... 10

**6. Module 4 : predictive_parser.h (LL(1) Parser)** ..... 11
**6.1 Grammar Definition** ..... 11
**6.2 FIRST and FOLLOW Set Computation** ..... 11
**FIRST Set Algorithm:** ..... 11
**FOLLOW Set Algorithm:** ..... 11
**6.3 LL(1) Table Construction** ..... 11
**6.4 Action Symbol Mechanism** ..... 12
**6.5 Panic-Mode Error Recovery** ..... 12
**7. Module 5 : lr_parser.h (Canonical LR(1) Parser)** ..... 12
**7.1 LR(1) Item and Item Sets** ..... 13
**7.2 ACTION and GOTO Table Construction** ..... 13
**7.3 Shift-Reduce Parse Algorithm** ..... 13
**7.4 Semantic Actions on Reduce** ..... 13
**8. Module 6 : symbol_table.h** ..... 14
**8.1 SymbolEntry Structure** ..... 14
**8.2 Hash Function – djb2** ..... 15
**8.3 Scope Management – Design A (Stack of Tables)** ..... 15
**8.4 Symbol Table Pretty-Print Format** ..... 15
**ID** ..... 15
**Name** ..... 15
**Kind** ..... 15
**Type** ..... 15
**Scope** ..... 15
**Line** ..... 15
**Offset** ..... 15
1 ..... 15
myProgram ..... 15
function ..... 15
void ..... 15
0 ..... 15
1 ..... 15
0 ..... 15

2 ..... 15
x ..... 15
variable ..... 15
integer ..... 15
1 ..... 15
2 ..... 15
0 ..... 15
3 ..... 15
y ..... 15
variable ..... 15
integer ..... 15
1 ..... 15
2 ..... 15
1 ..... 15
**8.5 Semantic Error Detection** ..... 15
**9. Module 7 : main.cpp (Pipeline Driver)** ..... 15
**9.1 runFile() Function** ..... 15
**9.2 Command-Line Interface** ..... 16
**9.3 Default Test Files** ..... 16
**10. Error Handling : ErrorHandler (common.h)** ..... 16
**10.1 CompilerError Structure** ..... 16
**10.2 Error Types by Phase** ..... 16
**10.3 Panic-Mode Recovery Summary** ..... 17
**11. Module Integration and Cross-Module Dependencies** ..... 17
**11.1 Data Flow Between Modules** ..... 17
**12. Requirements Alignment** ..... 18
**13. Conclusion** ..... 19

# 1. Introduction

This report documents the design, implementation, and integration of a complete compiler front-end for a Pascal language subset, developed as the final project for CS-471L: Compiler Construction Lab. The project integrates all major lab modules built throughout the semester (lexical analysis, recursive descent parsing, LL(1) predictive parsing, canonical LR(1) parsing, symbol table management, and error reporting) into a single unified compilation pipeline.

The compiler is implemented entirely in C++17 using header-only modules. Each component was designed to be independently testable before being wired together through the main pipeline driver (main.cpp). The system takes a Pascal source file as input and passes it sequentially through each compilation stage, printing detailed trace output at every step.

## 1.1 Project Objectives

- Implement a functional multi-phase compiler front-end for a Pascal subset.
- Integrate lexical analysis, multiple parsing strategies, symbol table, and error handling into a single pipeline.
- Build an Abstract Syntax Tree (AST) during recursive descent parsing.
- Perform semantic checks (duplicate declarations, undeclared identifiers) during parsing.
- Support panic-mode error recovery in all three parsers.
- Produce readable, structured trace output for each compilation phase.

## 1.2 Language Subset Supported

The Pascal language subset handled by this compiler includes the following constructs:

| **Category** | **Constructs** |
|---|---|
| Data Types | integer, real, array[N..M] of type |
| Declarations | VAR declarations, program-level and block-level |
| Control Flow | if-then-else, while-do |

| Category | Constructs |
|---|---|
| Subprograms | FUNCTION and PROCEDURE declarations with parameters |
| Expressions | Arithmetic (+, -, *, /), relational (<, >, =, <=, >=, <>), logical (and, or, not, div, mod) |
| Statements | Assignment (:=), compound (begin...end), procedure calls |
| Literals | Integer constants, real (floating-point) constants |
| Comments | {...} style block comments |

# 2. System Architecture and File Structure

The project is organized into seven header-only C++ files plus one driver source file. Each header implements one self-contained module. All modules share the common types and utilities defined in common.h.

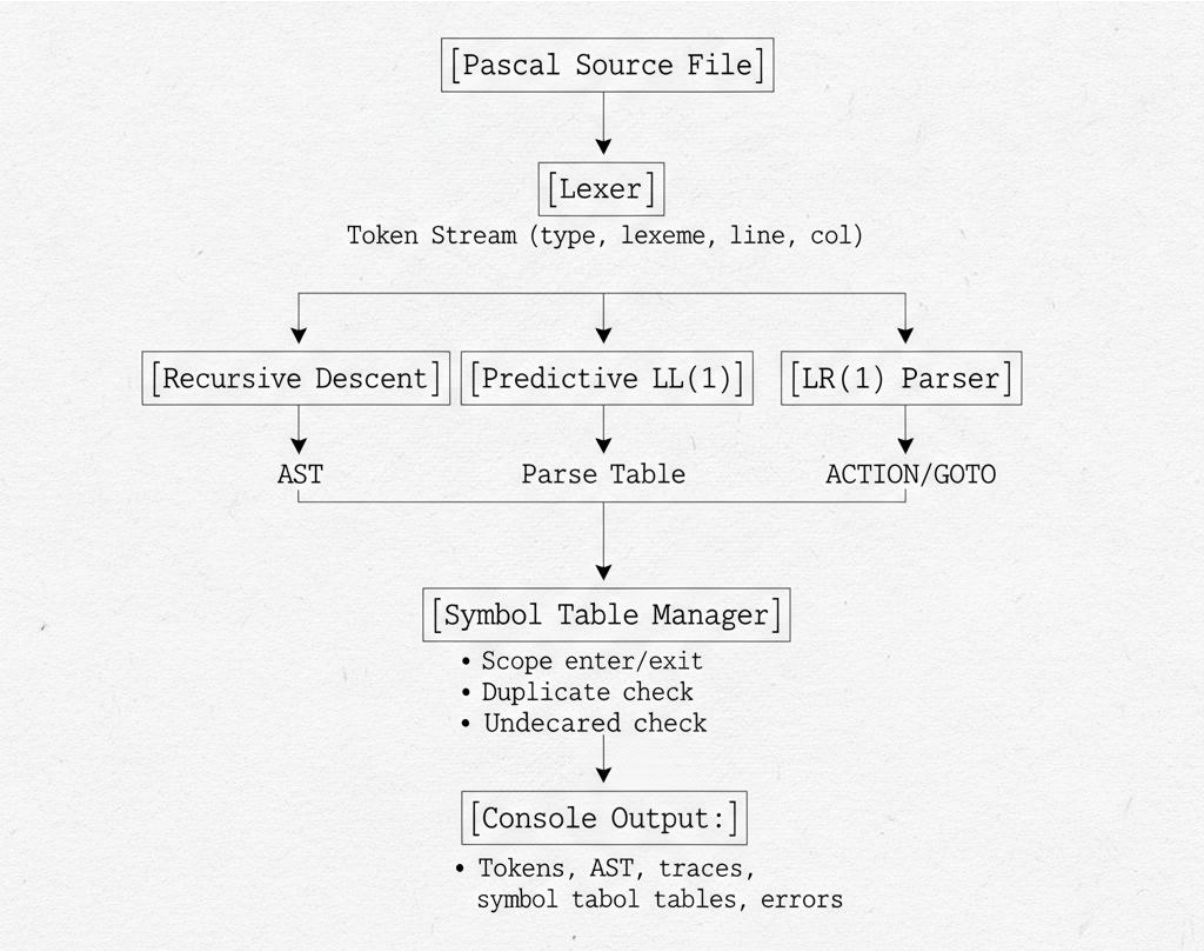
## 2.1 File-to-Module Mapping

| File | Module / Class | Role in Pipeline |
|---|---|---|
| common.h | Shared definitions | Token struct, ASTNode, ASTNodeKind enum, ErrorHandler, utility functions |
| lexer.h | Lexer | Double-buffered file scanner. Converts source file to token stream |
| recursive_descent_parser.h | RecursiveDescentParser | Top-down LL recursive parser. Builds AST. Integrates SymbolTableManager |

| File | Module / Class | Role in Pipeline |
|---|---|---|
| predictive_parser.h | PredictiveParser | Non-recursive LL(1) table-driven parser. FIRST/FOLLOW sets, LL(1) table, @action symbols |
| lr_parser.h | LRParser | Canonical LR(1) shift-reduce parser. Constructs item sets, ACTION/GOTO tables |
| symbol_table.h | SymbolTable, SymbolTableManager | Hash-table based symbol table with chained collision handling |
| main.cpp | Pipeline Driver | Wires all modules. Runs Lexer → RDP → Predictive → LR sequentially |

## 2.2 Compilation Pipeline

The full pipeline is shown below:




# 3. Module 1: common.h (Shared Definitions)

The common.h header serves as the central shared library for the entire project.

## 3.1 Token Structure

```
struct Token {
    string type;    // e.g. "ID", "num", "begin", "relop", ":="
    string lexeme;  // actual source text, e.g. "myVar", "42", "<="
    int line;       // 1-indexed source line
    int column;     // 1-indexed source column
};
```

## 3.2 ASTNode and ASTNodeKind

| ASTNodeKind | Meaning |
|---|---|
| PROGRAM | Root node. value = program name. |
| VAR_DECL | Variable declaration. |
| ASSIGN_STMT | Assignment statement. |
| WHILE_STMT | While loop. |
| IF_STMT | If statement. |
| BINARY_EXPR | Binary expression. |
| ID_NODE | Identifier leaf. |
| NUM_NODE | Numeric literal leaf. |
| BLOCK_STMT | Grouping node. |
| SUBPROGRAM_DECL | Function/procedure declaration. |
| EMPTY_NODE | Sentinel for epsilon. |

## 3.3 ErrorHandler

Collects CompilerError structs from all phases (phase name, line, column, message). print() outputs a summary table.

## 3.4 Utility Functions

- lowerText(s) – converts string to lowercase (case-insensitive keywords).
- tokenTypes(tokens) – extracts only type field from tokens.
- printTokenStream(tokens, out) – pretty-prints token table.

# 4. Module 2 : lexer.h (Lexical Analyzer)

Implements a double-buffered file scanner.

## 4.1 Double-Buffer Architecture

Two 4096-byte buffers. ensureLoaded(index) loads the appropriate file block. Minimizes disk I/O.

Key fields: buf[0], buf[1], blockStart[0], blockStart[1], absPos, fileLen, bufLenArr.

## 4.2 Scanning Methods

- scanIdentifier() – accumulates letters/digits, lowercases, checks keywords.
- scanNumber() – handles integers and reals.
- scanOperator() – handles :=, .., <=, >=, <>, +, -, *, /, punctuation.
- skipComment() – skips {...} comments, reports unterminated.

## 4.3 Keyword-to-Token Type Mapping

| Keyword | Token Type | Keyword | Token Type |
|---|---|---|---|
| program | PROGRAM | integer | integer |
| var | VAR | real | real |
| begin | begin | not | not |
| end | end | or | addop |
| if | IF | div, mod, and | mulop |
| function | FUNCTION | +, - | addop |
| procedure | PROCEDURE | *, / | mulop |
| array | ARRAY | < > = <= >= <> | relop |

# 5. Module 3 : recursive_descent_parser.h

Handwritten top-down parser. Builds AST and performs symbol table operations.

## 5.1 Grammar Coverage

| Non-terminal | Method | Returns |
|---|---|---|
| program | nt_program() | ASTNode |
| declarations | nt_declarations() | ASTNode |
| subprogram_declarations | nt_subprogram_declarations() | ASTNode |
| compound_statement | nt_compound_statement() | ASTNode |
| statement | nt_statement() | ASTNode |
| expression | nt_expression() | ASTNode |
| simple_expression | nt_simple_expression() | ASTNode |
| term | nt_term() | ASTNode |
| factor | nt_factor() | ASTNode |

## 5.2 Left-Recursion Elimination

**Original left-recursive grammar:**

simple_expression → simple_expression addop term | term

**Transformed to right-recursive:**

simple_expression → term simple_expression_p

simple_expression_p → addop term simple_expression_p | ε

## 5.3 AST Construction

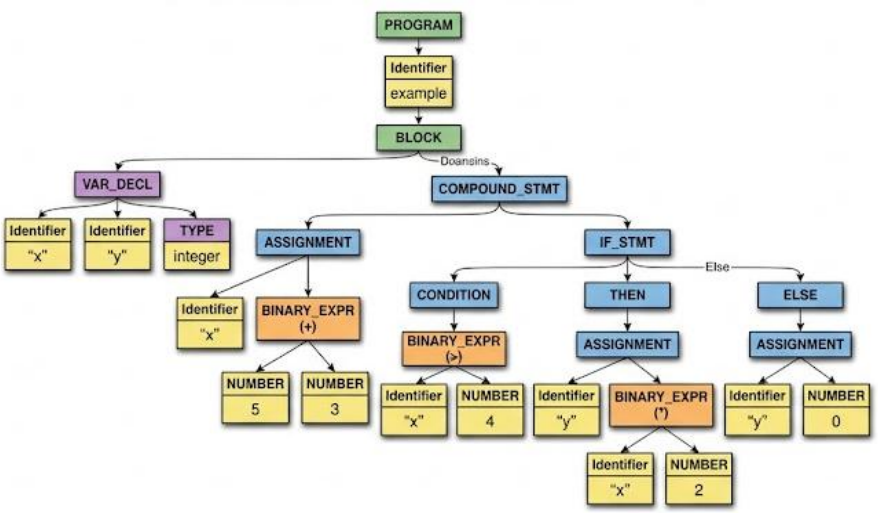
- PROGRAM node: up to 3 children (declarations, subprograms, compound statement).
- BINARY_EXPR: left and right operands.

- ASSIGN_STMT: LHS variable, RHS expression.
- IF_STMT: condition, then-branch, else-branch.
- WHILE_STMT: condition, body.

## 5.4 Example AST

For the program:

```
program example(input, output);
var x, y : integer;
begin
  x := 5 + 3;
  if x > 4 then y := x * 2
  else y := 0
end.
```




## 5.5 Symbol Table Integration

| Grammar Event | Semantic Action |
|---|---|
| program name parsed | beginScope() + insert(program_name, FUNCTION, VOID) |
| var declarations | insertNames(names, VARIABLE, type) |
| function/procedure head | insert(name, FUNCTION/PROCEDURE, VOID) + beginScope() |

| Grammar Event | Semantic Action |
|---|---|
| ID in expression | symbols.lookup(name) – error if undeclared |
| compound_statement ends | endScope() |

# 6. Module 4 : predictive_parser.h (LL(1) Parser)

Non-recursive table-driven parser. Manages explicit stack and LL(1) table.

## 6.1 Grammar Definition

Defined as vector of Production pairs inside defineGrammar().

## 6.2 FIRST and FOLLOW Set Computation

**FIRST Set Algorithm:**

- For each terminal t: FIRST(t) = {t}
- For each production A → X1 X2 ... Xk:
  - Add FIRST(X1) − {ε} to FIRST(A)
  - If ε ∈ FIRST(Xi), also add FIRST(Xi+1) − {ε}, and so on
  - If ε ∈ FIRST(Xi) for all i, add ε to FIRST(A)

**FOLLOW Set Algorithm:**

- Add $ to FOLLOW(start_symbol)
- For each production A → αBβ:
  - Add FIRST(β) − {ε} to FOLLOW(B)
  - If ε ∈ FIRST(β), add FOLLOW(A) to FOLLOW(B)
- For each production A → αB:
  - Add FOLLOW(A) to FOLLOW(B)

## 6.3 LL(1) Table Construction

For each production A → α:

- For each t in FIRST(α) − {ε}: table[(A, t)] = production index
- If ε ∈ FIRST(α): for each t in FOLLOW(A): table[(A, t)] = production index

### 6.4 Action Symbol Mechanism

Injects @action symbols into stack. When popped, calls executeAction().

| @Action Symbol | Effect |
|---|---|
| @program_scope_begin | Opens global scope |
| @subprogram_scope_begin | Opens scope for function/procedure |
| @scope_begin | Opens block scope |
| @scope_end | Closes scope, prints symbol table |
| @decl_var_start | Prepares variable declaration list |
| @decl_param_start | Prepares parameter list |
| @declare_program_name | Inserts program name |
| @declare_function_name | Inserts function name |
| @declare_procedure_name | Inserts procedure name |
| @decl_end | Flushes pending declarations |

### 6.5 Panic-Mode Error Recovery

- Add syntax error to ErrorHandler.
- Sync set = FOLLOW(current_nonterminal) ∪ {";", "end", "$"}.
- Skip tokens until sync set member found.
- Pop current nonterminal, continue.

## 7. Module 5 : lr_parser.h (Canonical LR(1) Parser)

Bottom-up shift-reduce parser. Constructs LR(1) item sets, ACTION and GOTO tables.

## 7.1 LR(1) Item and Item Sets

Item: (production index, dot position, lookahead terminal). Closure and goto operations build the canonical collection.

## 7.2 ACTION and GOTO Table Construction

| **Condition** | **Action** |
|---|---|
| [A → α . a β, b] in state i, goto(i,a)=j | ACTION[i,a] = shift j |
| [A → α ., a] in state i (A ≠ S') | ACTION[i,a] = reduce by A→α |
| [S' → S ., $] in state i | ACTION[i,$] = accept |
| goto(i,A) = j | GOTO[i,A] = j |

## 7.3 Shift-Reduce Parse Algorithm

1. stateStack = [0], symbolStack = [], ip = 0

2. s = top(stateStack), a = tokens[ip]

3. If ACTION[s,a] = shift j:

    push a, push j, ++ip

4. If ACTION[s,a] = reduce by A→β:

    pop |β| symbols and states

    executeSemanticAction(production)

    t = top(stateStack), push GOTO[t,A]

5. If ACTION[s,a] = accept: done

6. If error: panic-mode recovery

## 7.4 Semantic Actions on Reduce

| **Production Reduced** | **Semantic Action** |
|---|---|
| identifier_list → ID ... | Build names list |
| standard_type → integer/real | Propagate SymbolType |

| Production Reduced | Semantic Action |
|---|---|
| declarations → VAR id_list : type ; | Insert variables |
| parameter_list → id_list : type | Insert parameters |
| subprogram_head → FUNCTION ID ... | Insert function name |
| compound_statement → begin ... end | endScope() |
| factor → ID | lookup() identifier |

# 8. Module 6 : symbol_table.h

## 8.1 SymbolEntry Structure

| Field | Type | Description |
|---|---|---|
| id | int | Unique global identifier |
| name | string | Identifier lexeme |
| kind | SymbolKind | VARIABLE, FUNCTION, PROCEDURE, PARAMETER, etc. |
| type | SymbolType | INTEGER, REAL, VOID_TYPE, etc. |
| scopeLevel | int | 0 = global, 1 = nested, etc. |
| line/column | int | Source position |
| offset | int | Allocation offset |
| next | shared_ptr | Chain pointer for collisions |

## 8.2 Hash Function – djb2

unsigned long h = 5381;

for (char c : text) { h = ((h << 5) + h) + (unsigned char)c; }

return (int)(h % TABLE_SIZE);  // TABLE_SIZE = 211

## 8.3 Scope Management – Design A (Stack of Tables)

- beginScope(): allocates new SymbolTable, sets parent.
- endScope(): dumps current table, moves to parent.
- lookup(name): walks parent chain, returns first match.
- lookupCurrent(name): checks only current scope (duplicate detection).

## 8.4 Symbol Table Pretty-Print Format

| ID | Name | Kind | Type | Scope | Line | Offset |
|---|---|---|---|---|---|---|
| 1 | myProgram | function | void | 0 | 1 | 0 |
| 2 | x | variable | integer | 1 | 2 | 0 |
| 3 | y | variable | integer | 1 | 2 | 1 |

## 8.5 Semantic Error Detection

- **Duplicate declaration**: lookupCurrent() finds existing entry → add error.
- **Undeclared identifier**: lookup() returns nullptr → add error.

# 9. Module 7 : main.cpp (Pipeline Driver)

### 9.1 runFile() Function

1. Instantiate Lexer, tokenize file → token stream.
2. Print token stream table.
3. Instantiate RecursiveDescentParser, parse, print AST.
4. Instantiate PredictiveParser, parse, print LL(1) trace.
5. If --with-lr flag: instantiate LRParser, parse, print LR trace.

### 9.2 Command-Line Interface

| Flag | Effect |
|---|---|
| --with-lr | Enables LR(1) parser stage |
| --dump-docs | Exports FIRST/FOLLOW, LL(1) table, grammar BNF, LR tables to submission/docs/ |
| [filename ...] | Pascal source files to compile (default: test files) |

### 9.3 Default Test Files

- valid_nested.pascal – valid program with nested scopes.
- duplicate_same_scope.pascal – duplicate declaration test.
- undeclared_variable.pascal – undeclared identifier test.
- shadowing_valid.pascal – scope shadowing test.
- mixed_errors.pascal – syntax + semantic errors.

# 10. Error Handling : ErrorHandler (common.h)

## 10.1 CompilerError Structure

```
struct CompilerError {
    string phase;    // "Lexical", "Syntax", or "Semantic"
    int line;        // source line number (1-indexed)
    int column;      // source column number (1-indexed)
    string message;  // human-readable description
};
```

## 10.2 Error Types by Phase

| Phase | Module | Example Errors |
|---|---|---|
| Lexical | Lexer | Unterminated comment, unknown character |

| Phase | Module | Example Errors |
|---|---|---|
| Syntax | RDP | Expected 'begin' but got 'ID' |
| Syntax | PredictiveParser | No LL(1) table entry |
| Syntax | LRParser | No ACTION entry for state N and token X |
| Semantic | All parsers | Duplicate declaration, undeclared variable |

### 10.3 Panic-Mode Recovery Summary

| Parser | Recovery Strategy |
|---|---|
| RecursiveDescentParser | Reports error, returns false – parse halts at first error |
| PredictiveParser | Sync set = FOLLOW(top) ∪ {";","end","$"} – skip tokens, pop, continue |
| LRParser | Sync tokens = {";","end","ELSE","THEN","DO","$"} – skip until ACTION entry exists |

## 11. Module Integration and Cross-Module Dependencies

### 11.1 Data Flow Between Modules

| From | To | Data Transferred |
|---|---|---|
| Lexer | All 3 parsers | vector<Token> (token stream + $) |
| RDP | main.cpp | shared_ptr<ASTNode> root |

| From | To | Data Transferred |
|---|---|---|
| SymbolTableManager | All parsers (embedded) | Each parser has its own instance – no sharing |
| ErrorHandler | All modules | Passed by reference, collects errors |

## 12. Requirements Alignment

| Lab Requirement | Implemented In | Notes |
|---|---|---|
| Lexical Analyzer | lexer.h | Double-buffered DFA |
| Double buffering | lexer.h – ensureLoaded() | Two 4096-byte buffers |
| Token stream (type, lexeme, line, col) | common.h – Token | All four fields |
| Recursive Descent Parser | recursive_descent_parser.h | One method per non-terminal |
| AST construction | recursive_descent_parser.h + common.h | 11 node kinds |
| Non-recursive LL(1) Predictive Parser | predictive_parser.h | Stack-driven, FIRST/FOLLOW, table |
| FIRST and FOLLOW computation | predictive_parser.h + lr_parser.h | Fixed-point algorithms |
| LR Parser (Canonical LR(1)) | lr_parser.h | Item sets, ACTION/GOTO |

| Lab Requirement | Implemented In | Notes |
|---|---|---|
| Symbol Table with scope management | symbol_table.h | Design A: stack of hash tables |
| Duplicate declaration detection | SymbolTableManager::insert() | lookupCurrent() before insert |
| Undeclared identifier detection | SymbolTableManager::lookup() | Full chain lookup |
| Error Handler | common.h – ErrorHandler | Phase, line, column, message |
| Panic-mode error recovery | predictive_parser.h + lr_parser.h | FOLLOW-based synchronization |
| Integrated pipeline driver | main.cpp – runFile() | Sequential execution |
| Documentation output | --dump-docs flag | FIRST/FOLLOW, LL(1) table, LR tables |

# 13. Conclusion

This project successfully implements a complete compiler for a Pascal language subset in C++17. All six required modules  Lexer, Recursive Descent Parser, LL(1) Predictive Parser, Canonical LR(1) Parser, Symbol Table, and Error Handler  were implemented from scratch and integrated into a working pipeline.

The most technically demanding components were the LR(1) parser (closure/goto over LR(1) item sets, ACTION/GOTO tables) and the Predictive Parser's action symbol mechanism. The symbol table's scope chain correctly handles shadowing, duplicate detection, and cross-scope lookups.

The pipeline produces detailed trace output at every stage  lexer character traces, RDP call stack traces, LL(1) step-by-step tables, LR shift-reduce traces, and symbol table dumps making it a valuable tool for learning compiler construction concepts.